

Apply filters to SQL queries

Project description

I am security professional at a large organization. Part of my job is to investigate security issues to help keep the system secure. I recently discovered some potential security issues that involve login attempts and employee machines.

I will be examining the organization's data in the **employees** and **log_in_attempts** tables using SQL filters to retrieve records from different datasets and investigate the potential security issues. SQL allows us to efficiently filter out data based on conditions. It allows for the removal of noise or irrelevant data for the scenario that is being examined.

To get an idea of the data in the dataset and the headers, the DESC log_in_attempts command has been executed to provide a brief description of the table that is being examined.

```
MariaDB [organization]> DESC log_in_attempts
-> ;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| event_id   | int(11)       | NO   | PRI | NULL     |      |
| username   | varchar(16)   | NO   |     | NULL     |      |
| login_date | date          | NO   |     | NULL     |      |
| login_time | time          | NO   |     | NULL     |      |
| country    | varchar(16)   | NO   |     | NULL     |      |
| ip_address | varchar(16)   | NO   |     | NULL     |      |
| success    | tinyint(1)    | YES  |     | NULL     |      |
+-----+-----+-----+-----+-----+-----+
7 rows in set (0.050 sec)

MariaDB [organization]> 
```

From this description, we get a preliminary view of the table headers and the data type of the entries.

Retrieve after hours failed login attempts

The team is investigating **failed** login attempts made after business hours. To identify these attempts made **after 18:00**, the following command has been executed;

```
MariaDB [organization]> SELECT * FROM log_in_attempts WHERE login_time > '18:00' AND success = FALSE;
```

event_id	username	login_date	login_time	country	ip_address	success
2	apatel	2022-05-10	20:27:27	CAN	192.168.205.12	0
18	pwashing	2022-05-11	19:28:50	US	192.168.66.142	0
20	tshah	2022-05-12	18:56:36	MEXICO	192.168.109.50	0
28	aestrada	2022-05-09	19:28:12	MEXICO	192.168.27.57	0
34	drosas	2022-05-11	21:02:04	US	192.168.45.93	0
42	cgriffin	2022-05-09	23:04:05	US	192.168.4.157	0
52	cjackson	2022-05-10	22:07:07	CAN	192.168.58.57	0
69	wjaffrey	2022-05-11	19:55:15	USA	192.168.100.17	0
82	abernard	2022-05-12	23:38:46	MEX	192.168.234.49	0
87	apatel	2022-05-08	22:38:31	CANADA	192.168.132.153	0
96	ivelasco	2022-05-09	22:36:36	CAN	192.168.84.194	0
104	asundara	2022-05-11	18:38:07	US	192.168.96.200	0
107	bisles	2022-05-12	20:25:57	USA	192.168.116.187	0
111	aestrada	2022-05-10	22:00:26	MEXICO	192.168.76.27	0
127	abellmas	2022-05-09	21:20:51	CANADA	192.168.70.122	0
131	bisles	2022-05-09	20:03:55	US	192.168.113.171	0
155	cgriffin	2022-05-12	22:18:42	USA	192.168.236.176	0
160	jclark	2022-05-10	20:49:00	CANADA	192.168.214.49	0
199	yappiah	2022-05-11	19:34:48	MEXICO	192.168.44.232	0

```
19 rows in set (0.016 sec)

MariaDB [organization]>
```

The AND operator has been used to ensure that both conditions are met.

Retrieve login attempts on specific dates

The team is investigating a suspicious event that occurred on '2022-05-09'. Need to retrieve all login attempts that occurred on this day and the day before ('2022-05-08'). The following command has been used.

```
MariaDB [organization]> SELECT *
-> FROM log_in_attempts
-> WHERE login_date = '2022-05-09' OR login_date = '2022-05-08';
```

event_id	username	login_date	login_time	country	ip_address	success
1	jrafael	2022-05-09	04:56:27	CAN	192.168.243.140	1
3	dkot	2022-05-09	06:47:41	USA	192.168.151.162	1
4	dkot	2022-05-08	02:00:39	USA	192.168.178.71	0
8	bisles	2022-05-08	01:30:17	US	192.168.119.173	0
12	dkot	2022-05-08	09:11:34	USA	192.168.100.158	1
15	luamwet	2022-05-08	17:17:26	USA	192.168.182.51	0

The OR operator ensures that entries that satisfy both or one of the conditions will be output.

Retrieve login attempts outside of Mexico

Now the team is investigating logins that did not originate in Mexico, and I need to find this information. In the country column, Mexico has been written as 'MEX' and 'MEXICO'. To make the query, the LIKE operator and the matching pattern 'MEX%' can be used to retrieve all

instances of MEX followed by other alphabets LIKE has been used to replace = operator, allowing us to use the % wildcard. I will also use the NOT operator to get all log in attempts that did not originate from Mexico.

```
MariaDB [organization]> SELECT * FROM log_in_attempts WHERE NOT country LIKE 'MEX%';
```

event_id	username	login_date	login_time	country	ip_address	success
1	jrafael	2022-05-09	04:56:27	CAN	192.168.243.140	1
2	apatel	2022-05-10	20:27:27	CAN	192.168.205.12	0
3	dkot	2022-05-09	06:47:41	USA	192.168.151.162	1
4	dkot	2022-05-08	02:00:39	USA	192.168.178.71	0
5	jrafael	2022-05-11	03:05:59	CANADA	192.168.86.232	0
7	eraab	2022-05-11	01:45:14	CAN	192.168.170.243	1

Retrieve employees in Marketing

For the next few tasks, the employees table has been used. Hence, the DESC employees command has been run to view information about the table.

```
MariaDB [organization]> DESC employees
-> ;
```

Field	Type	Null	Key	Default	Extra
employee_id	int(11)	NO	PRI	NULL	
device_id	varchar(16)	YES		NULL	
username	varchar(16)	NO		NULL	
department	varchar(32)	NO		NULL	
office	varchar(32)	NO		NULL	

5 rows in set (0.001 sec)

The team is updating employee machines, and I need to obtain the information about employees in the 'Marketing' department who are located in all offices in the East building (such as 'East-170' or 'East-320').

```
MariaDB [organization]> SELECT *
-> FROM employees
-> WHERE department = 'Marketing' AND office LIKE 'East%';
```

employee_id	device_id	username	department	office
1000	a320b137c219	elarson	Marketing	East-170
1052	a192b174c940	jdarosa	Marketing	East-195
1075	x573y883z772	fbautist	Marketing	East-267
1088	k865l965m233	rgosh	Marketing	East-157
1103	NULL	randerss	Marketing	East-460
1156	a184b775c707	dellery	Marketing	East-417
1163	h679i515j339	cwilliam	Marketing	East-216

```
7 rows in set (0.001 sec)
```

Retrieve employees in Finance or Sales

Now, the team needs to perform a different update to the computers of all employees in the Finance or the Sales department, and I need to locate information on these employees.

```
MariaDB [organization]> SELECT *
-> FROM employees
-> WHERE department = 'Finance' OR department = 'Sales';
```

employee_id	device_id	username	department	office
1003	d394e816f943	sgilmore	Finance	South-153
1007	h174i497j413	wjaffrey	Finance	North-406
1008	i858j583k571	abernard	Finance	South-170
1009	NULL	lrodriqu	Sales	South-134
1010	k242l212m542	jlsansky	Finance	South-109
1011	l748m120n401	drozas	Sales	South-292
1015	p611q262r945	jsoto	Finance	North-271
1017	r550s824t230	jclark	Finance	North-188
1018	s310t540u653	abellmas	Finance	North-403
1022	w237x430y567	arusso	Finance	West-465
1024	y976z753a267	iuduike	Sales	South-215

Retrieve all employees not in IT

The team needs to make one more update. This update was already made to employee computers in the Information Technology department. The team needs information about employees who are not in that department.

```
MariaDB [organization]> SELECT * FROM employees WHERE NOT department = "Information Technology";
```

employee_id	device_id	username	department	office
1000	a320b137c219	elarson	Marketing	East-170
1001	b239c825d303	bmoreno	Marketing	Central-276
1002	c116d593e558	tshah	Human Resources	North-434
1003	d394e816f943	sgilmore	Finance	South-153
1004	e218f877g788	eraab	Human Resources	South-127
1005	f551g340h864	gesparza	Human Resources	South-366
1007	h174i497j413	wjaffrey	Finance	North-406
1008	i858j583k571	abernard	Finance	South-170
1009	NULL	lrodriqu	Sales	South-134

Summary

The tasks mentioned have been completed using operators such as, AND, OR, NOT and LIKE as well as the % wildcard. These operators and wildcards are invaluable in SQL and provide efficiency in analysing data.